K8S-14: Kubernetes Deployment Guide

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 14 of 14 | **Type:** LAB | **Created:** April 2026 | **Last Updated:** 07/24/2026

Overview

Step-by-step guide to deploy Dynatrace monitoring on Kubernetes with all recommended production settings. Covers operator installation, DynaKube configuration, metadata enrichment, build propagation, segments, and verification.

This is a hands-on deployment notebook. Each section includes commands to run and queries to verify. Follow the sections in order for a complete production deployment.

Table of Contents

1. Prerequisites

Requirement	Details
Kubernetes Cluster	Version 1.24+ with `kubectl` access and cluster-admin permissions
Helm	Version 3.x installed locally
Dynatrace Tenant	SaaS environment with admin access
Network	Outbound 443 to `*.live.dynatrace.com` and `*.apps.dynatrace.com`
Knowledge	Completed K8S-01 (Fundamentals) and K8S-02 (DynaKube Deployment)

Verify Your Environment

Run these commands to confirm your cluster is ready:

```
# Verify Kubernetes version (must be 1.24+)
kubectl version --short

# Verify Helm version (must be 3.x)
helm version --short

# Verify cluster access
kubectl get nodes
```

2. Create Access Tokens

The Dynatrace Operator requires two tokens: an **API token** for cluster communication and a **data ingest token** for sending telemetry.

API Token

1. Navigate to **Access Tokens** > **Generate new token**

2. Select the template: **Kubernetes: Dynatrace Operator**
3. This template includes the required scopes:

Scope	Purpose
`InstallerDownload`	Download OneAgent and ActiveGate binaries
`activeGateTokenManagement.create`	Create ActiveGate auth tokens
`entities.read`	Read entity topology
`settings.read`	Read environment settings
`settings.write`	Write environment settings

Data Ingest Token

1. Navigate to **Access Tokens** > **Generate new token**
2. Select the template: **Kubernetes: Data Ingest**
3. This template includes the required scopes:

Scope	Purpose
`metrics.ingest`	Send Kubernetes and Prometheus metrics
`logs.ingest`	Send container and pod logs
`openTelemetryTrace.ingest`	Send distributed traces

> **Important:** Always use the predefined templates. Manually selecting scopes risks missing required permissions, which causes silent failures that are difficult to diagnose.

Save Your Tokens

Copy both tokens immediately after generation. You cannot retrieve them later.

```
# Store tokens as environment variables for the next steps
export API_TOKEN="dt0c01.XXXXXXXX.YYYYYYYYYYYYYYYY"
export DATA_INGEST_TOKEN="dt0c01.XXXXXXXX.ZZZZZZZZZZZZZZZZ"
```

3. Install Dynatrace Operator

Step 1: Install the Operator via Helm

```
helm install dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --create-namespace \
  --namespace dynatrace \
  --atomic
```

The `--atomic` flag ensures the installation is rolled back automatically if any component fails to start.

Step 2: Create the Token Secret

```
kubectl -n dynatrace create secret generic dynakube \
  --from-literal=apiToken=$API_TOKEN \
  --from-literal=dataIngestToken=$DATA_INGEST_TOKEN
```

Step 3: Verify Operator Pods

```
kubectl get pods -n dynatrace
```

Expected output:

```
NAME                                 READY   STATUS    RESTARTS   AGE
dynatrace-operator-xxxxxxxxxx-xxxxx  1/1     Running   0          30s
dynatrace-webhook-xxxxxxxxxx-xxxxx   1/1     Running   0          30s
```

> **Note:** The operator and webhook pods must both be `Running` before applying the DynaKube CR. If either pod is in `CrashLoopBackOff`, check the logs with `kubectl logs -n dynatrace <pod-name>`.

4. Configure DynaKube CR (Production

Recommended)

The DynaKube custom resource defines how Dynatrace monitors your cluster. The configuration below represents production-recommended settings with all key features enabled.

Complete DynaKube YAML

Save this as `dynakube.yaml` and customize the placeholder values:

```yaml
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
  annotations:
    # Fail pod startup if injection fails — surfaces problems immediately
    feature.dynatrace.com/injection-failure-policy: "fail"
    # Opt-in injection — only labeled namespaces get instrumented
    feature.dynatrace.com/automatic-injection: "false"
    # Enable build label propagation for Release Inventory
    feature.dynatrace.com/label-version-detection: "true"
    # Extend CSI mount timeout for cloud providers with slow volume attachment
    feature.dynatrace.com/max-csi-mount-timeout: "15m"
spec:
  apiUrl: https://&lt;ENVIRONMENT_ID&gt;.live.dynatrace.com/api

  # Network zone — isolates traffic per cluster
  networkZone: &lt;cluster-name&gt;

  # Metadata enrichment — flows K8s labels into all telemetry
  metadataEnrichment:
    enabled: true

  oneAgent:
    # Host group — scopes dashboards and alerts per cluster
    hostGroup: &lt;cluster-name&gt;
    cloudNativeFullStack:
      # Opt-in namespace selector
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"

      # Host properties for entity-level context
      args:
        - --set-host-property=environment=production
        - --set-host-property=team=platform
        - --set-host-property=cost-center=infrastructure

      # Tolerate control plane nodes
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # ActiveGate — K8s API monitoring + routing
  activeGate:
```

```
    capabilities:
      - routing
      - kubernetes-monitoring
      - metrics-ingest
    replicas: 2
    resources:
      requests:
        cpu: 500m
        memory: 1Gi
      limits:
        cpu: 2000m
        memory: 2Gi
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels:
            dynatrace.com/component: activegate

  # Log monitoring
  logMonitoring: {}
```

Apply the DynaKube CR

```
# Replace placeholders, then apply
kubectl apply -f dynakube.yaml
```

Setting Explanations

Setting	Value	Why It Matters
`injection-failure-policy: "fail"`	Fail pod startup on injection error	Surfaces misconfiguration immediately instead of running uninstrumented
`automatic-injection: "false"`	Opt-in injection via namespace labels	Prevents accidental injection into system namespaces

Setting	Value	Why It Matters
`label-version-detection: "true"`	Propagate `app.kubernetes.io/version`	Populates Release Inventory for deployment tracking
`max-csi-mount-timeout: "15m"`	Extended CSI driver timeout	Prevents pod failures on cloud providers with slow EBS/disk attachment
`networkZone`	Per-cluster zone	Isolates ActiveGate traffic; required for multi-cluster
`metadataEnrichment.enabled`	`true`	Flows K8s labels/annotations into all telemetry signals
`cloudNativeFullStack`	Full-stack mode	Full code-level visibility with automatic injection
`namespaceSelector`	`dt-monitoring: "true"`	Only labeled namespaces get OneAgent injection
`hostGroup`	Per-cluster name	Groups hosts for scoped alerting and dashboards
`args: --set-host-property`	Custom properties	Adds environment, team, cost-center to every host entity
`tolerations`	Control plane toleration	Ensures DaemonSet runs on all nodes including control plane
`activeGate.replicas: 2`	High availability	Survives single-pod failure; required for production

Setting	Value	Why It Matters
`topologySpreadConstraints`	Zone-aware scheduling	Distributes ActiveGate pods across availability zones
`capabilities: routing`	ActiveGate routing	Routes OneAgent traffic through ActiveGate
`capabilities: kubernetes-monitoring`	K8s API integration	Enables cluster-level monitoring, events, and workload data
`capabilities: metrics-ingest`	Prometheus scraping	Enables annotation-based Prometheus metric collection
`logMonitoring: {}`	Log collection enabled	Collects container stdout/stderr logs

5. Label Namespaces for Monitoring

Because the DynaKube uses `automatic-injection: "false"` with a `namespaceSelector`, only namespaces labeled with `dt-monitoring=true` will receive OneAgent injection.

Label Application Namespaces

```
# Label each application namespace
kubectl label namespace &lt;app-namespace-1&gt; dt-monitoring=true
kubectl label namespace &lt;app-namespace-2&gt; dt-monitoring=true

# Verify labeled namespaces
kubectl get namespaces -l dt-monitoring=true
```

Namespaces to NOT Label

These system namespaces should **never** receive the `dt-monitoring=true` label:

Namespace	Reason
`kube-system`	Core Kubernetes components — injection can cause instability
`kube-public`	Cluster info namespace — no application workloads
`kube-node-lease`	Node heartbeat namespace — no application workloads
`cert-manager`	Certificate management — injection interferes with TLS operations
`dynatrace`	Dynatrace's own namespace — self-monitoring is handled internally
`istio-system`	Service mesh control plane — injection conflicts with Envoy sidecars

> **Tip:** After labeling, trigger a rolling restart of deployments in those namespaces to activate injection: `kubectl rollout restart deployment -n <namespace>`.

6. Configure Telemetry Enrichment Rules

Telemetry enrichment flows Kubernetes labels and annotations into all telemetry signals (logs, metrics, traces, events). This enables filtering, cost allocation, and IAM scoping.

Settings Path

Settings > **Cloud and Virtualization** > **Kubernetes Telemetry Enrichment**

Schema ID: `builtin:kubernetes.generic.metadata.enrichment`

Recommended Enrichment Rules

#	Metadata Type	Key	Mapped To	Purpose
1	Namespace label	`team`	`dt.security_context`	Enables record-level IAM — teams see only their own data

#	Metadata Type	Key	Mapped To	Purpose
2	Namespace label	`cost-center`	`dt.cost.costcenter`	Enables FinOps cost allocation per team
3	Namespace label	`app.kubernetes.io/part-of`	`dt.cost.product`	Enables product-level cost attribution

Label Your Namespaces

The enrichment rules map **existing** Kubernetes labels to Dynatrace fields. Ensure your namespaces carry these labels:

```
# Add organizational labels to namespaces
kubectl label namespace &lt;app-namespace&gt; team=backend
kubectl label namespace &lt;app-namespace&gt; cost-center=ecommerce
kubectl label namespace &lt;app-namespace&gt; app.kubernetes.io/part-of=checkout-platform

# Verify labels
kubectl get namespace &lt;app-namespace&gt; --show-labels
```

Propagation Timing

Action	Propagation Time
New enrichment rule created	Up to 45 minutes
Namespace label added	Immediate (next telemetry cycle)
Pod restart after enrichment	Immediate

> **Important:** If you create enrichment rules after DynaKube is already deployed, existing pods must be restarted to pick up the enriched metadata. Run `kubectl rollout restart deployment -n <namespace>` for affected namespaces.

> **See also:** K8S-10 (Metadata Telemetry Enrichment) for a deep dive into enrichment methods, file locations, and advanced use cases.

7. Configure Build Propagation & Release Detection

Build propagation connects your Kubernetes deployment metadata to the Dynatrace Release Inventory. This gives you deployment tracking, version comparison, and release health monitoring.

What Is Already Enabled

The DynaKube in Section 4 includes:

```
annotations:
  feature.dynatrace.com/label-version-detection: "true"
```

This tells the Dynatrace Operator to read version information from standard Kubernetes labels.

Required Pod Labels

Your pods (or the Deployments/StatefulSets that create them) must carry these standard labels:

Label	Purpose	Example
`app.kubernetes.io/version`	Version string	`1.4.2` , `2025.03.15-hotfix`
`app.kubernetes.io/part-of`	Application group	`checkout-platform`
`app.kubernetes.io/name`	Component name	`payment-service`

Example Deployment snippet:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  labels:
    app.kubernetes.io/name: payment-service
    app.kubernetes.io/part-of: checkout-platform
    app.kubernetes.io/version: "1.4.2"
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: payment-service
        app.kubernetes.io/part-of: checkout-platform
        app.kubernetes.io/version: "1.4.2"
```

Optional: Custom Release Mapping

If your deployment model uses non-standard labels, add namespace annotations to override the defaults:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: checkout
  annotations:
    mapping.release.dynatrace.com/version: "metadata.labels['app-version']"
    mapping.release.dynatrace.com/product: "metadata.labels['product-name']"
    mapping.release.dynatrace.com/stage: "metadata.namespace"
```

Verify in Dynatrace

After deploying workloads with the required labels:

1. Navigate to **Releases** in the Dynatrace UI
2. Confirm your services appear with version information
3. Deploy a new version and verify the Release Inventory updates

8. Create Segments

Segments are the Gen3 replacement for Management Zones. They provide scoped views of your environment using enriched metadata fields.

Settings Path

Settings > **Ownership & Organization** > **Segments**

Recommended Segments

Create segments based on the enrichment rules configured in Section 6:

Segment Name	Filter	Use Case
Per-team: `Team: Backend`	`dt.security_context == "backend"`	Team-scoped dashboards and alerts
Per-team: `Team: Frontend`	`dt.security_context == "frontend"`	Team-scoped dashboards and alerts
Per-cluster: `Cluster: Production`	`k8s.cluster.name == "prod-us-east-1"`	Cluster-level views
Per-namespace: `Namespace: Checkout`	`k8s.namespace.name == "checkout"`	Application-scoped monitoring
Per-environment: `Environment: Staging`	`k8s.cluster.name == "staging-us-east-1"`	Environment isolation

Creating a Segment via API

```
curl -X POST "https://&lt;ENVIRONMENT_ID&gt;.apps.dynatrace.com/platform/classic/environment-api/v2/settings/objects" \
  -H "Authorization: Api-Token &lt;TOKEN&gt;" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:ownership.segments",
    "scope": "environment",
    "value": {
      "name": "Team: Backend",
      "description": "All telemetry owned by the backend team",
      "variables": {"type": "query", "value": "fetch logs | filter dt.security_context == \\"backend\\""}
    }
  }]'
```

> **Note:** Segments replace Management Zones in Dynatrace Gen3. Unlike Management Zones, segments are dynamic and work across all Grail data — not just entity-based data.
>
> **See also:** ORGNZ-06 (Segments & Data Access) for comprehensive segment design patterns.

9. Verify Deployment

Run these DQL queries to confirm every component is working correctly.

9.1 Verify Dynatrace Components Are Running

```
// Verify Dynatrace components are running
fetch logs, from:-1h
| filter k8s.namespace.name == "dynatrace"
| summarize count = count(), by:{k8s.pod.name}
| sort count desc
| limit 20
```

Expected: You should see pods for `dynatrace-operator`, `dynatrace-webhook`, `dynakube-oneagent-*`, and `dynakube-activegate-*`.

9.2 Verify Host Monitoring

```
// Check all monitored hosts
fetch dt.entity.host
| fieldsKeep entity.name, host_group, state
| sort entity.name asc

// Smartscape note (dt.entity.* is deprecated but still functional): state is a classic-only
// field with no Smartscape node equivalent (Smartscape expresses liveness via node lifetime).
// Keep the classic query above; host_group maps to the dt.host_group.id field on HOST nodes.
```

Expected: Every node in your cluster should appear with the `host_group` matching your `<cluster-name>` value from the DynaKube.

9.3 Verify Container Metrics Are Flowing

```
// Top containers by CPU
timeseries avgCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h, by:{k8s.container.name, k8s.namespace.name}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Expected: Container CPU metrics from your labeled namespaces. If this returns no data, verify namespace labels and wait for data to propagate (up to 5 minutes).

9.4 Verify Metadata Enrichment

```
// Check enrichment is working — dt.security_context should be populated
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

Expected: Logs grouped by the `team` label you assigned to namespaces. If empty, verify enrichment rules are configured (Section 6) and wait up to 45 minutes for propagation.

9.5 Verify Build Propagation (Release Inventory)

```
// Check Release Inventory population
fetch dt.entity.process_group_instance
| filter isNotNull(softwareVersion)
| fieldsKeep entity.name, softwareVersion
| limit 10

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "PROCESS"
//   | filter isNotNull(softwareVersion)
//   | fieldsKeep name, softwareVersion
//   | limit 10
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
```

Expected: Process groups with version information from `app.kubernetes.io/version` labels. If empty, verify your pods carry the required labels (Section 7).

9.6 Verify ActiveGate Health

```
// ActiveGate pod CPU — should show 2 pods across zones
timeseries agCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
  filter:{k8s.namespace.name == "dynatrace" and
matchesValue(k8s.container.name, "*activegate*")},
  by:{k8s.pod.name}
| fieldsAdd avgCpu = arrayAvg(agCpu)
| fields k8s.pod.name, avgCpu
```

Expected: Two ActiveGate pods with CPU metrics. If only one pod appears, check the topology spread constraints and verify your cluster has multiple availability zones.

10. Monitoring-the-Monitoring

After deployment, set up ongoing health checks for the Dynatrace components themselves.

10.1 OneAgent Memory Usage per Node

```
// OneAgent memory consumption per node
timeseries oaMem = avg(dt.kubernetes.container.memory_working_set), from:-1h,
  filter:{k8s.namespace.name == "dynatrace" and
matchesValue(k8s.container.name, "*oneagent*")},
  by:{k8s.pod.name}
| fieldsAdd avgMemMB = arrayAvg(oaMem) / 1048576
| sort avgMemMB desc
```

Healthy range: 300-800 MB per OneAgent pod. If consistently above 1 GB, check for excessive process groups or deep monitoring of high-cardinality applications.

10.2 Dynatrace Component Failures

```
// K8s events for OOMKilled, CrashLoopBackOff in dynatrace namespace
fetch events, from:-24h
| filter k8s.namespace.name == "dynatrace"
| filter event.kind == "K8S_EVENT"
| fields timestamp, event.name, event.kind, k8s.pod.name
| sort timestamp desc
| limit 30
```

Expected: No `OOMKilled` or `CrashLoopBackOff` events. If present, increase resource limits in the DynaKube CR.

10.3 Metric Data Gaps

```
// Check for gaps in host CPU metrics (indicates OneAgent interruptions)
timeseries hostCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd dataPoints = arraySize(hostCpu)
| fieldsAdd expectedPoints = 72
| fieldsAdd completeness = toDouble(dataPoints) / toDouble(expectedPoints) *
100.0
| filter completeness &lt; 95.0
| fields dt.entity.host, completeness, dataPoints
| sort completeness asc
```

Expected: All hosts at 100% completeness. Hosts below 95% indicate OneAgent restarts, pod evictions, or network interruptions to the Dynatrace backend.

11. Recommended Next Steps

With the base deployment complete, these additional configurations maximize the value of your Dynatrace Kubernetes monitoring:

Priority	Configuration	What It Enables	Reference
High	OpenPipeline log routing	Route logs to buckets by namespace, set retention tiers	OPLOGS series
High	Custom Grail buckets	Separate retention for prod vs. non-prod, compliance data	ORGNZ-02
High	anomaly detectors	Automated baseline alerting for CPU, memory, pod restarts	K8S-09
Medium	Kubernetes dashboards	Cluster health overview, workload summary, namespace drill-down	DASH series
Medium	Alerting workflows	detected problem → Slack, Teams, PagerDuty, Jira	WFLOW series
Medium	Prometheus scraping	Collect custom app metrics via pod annotations	K8S-13
Low	Multi-cluster federation	Unified view across dev/staging/prod clusters	K8S-04
Low	GitOps DynaKube management	Version-control DynaKube CR with ArgoCD or Flux	K8S-03

12. Summary

Deployment Checklist

Step	Status	Notes
1. Prerequisites verified	[ ]	K8s 1.24+, Helm 3.x, network access
2. Access tokens created	[ ]	API token + Data Ingest token via templates
3. Operator installed	[ ]	Helm chart + token secret

Step	Status	Notes
4. DynaKube applied	[ ]	v1beta6 with all production settings
5. Namespaces labeled	[ ]	`dt-monitoring=true` on app namespaces
6. Enrichment rules configured	[ ]	team, cost-center, part-of
7. Build labels added	[ ]	`app.kubernetes.io/version` on pods
8. Segments created	[ ]	Per-team, per-cluster, per-namespace
9. Deployment verified	[ ]	All DQL verification queries passing
10. Monitoring-the-monitoring	[ ]	OneAgent health, AG health, data gaps

Key Takeaways

Concept	Recommendation
Injection model	Opt-in via `namespaceSelector` — never inject into system namespaces
Failure policy	`fail` — surface problems immediately, do not run uninstrumented
ActiveGate	2 replicas with zone-aware topology spread
Metadata enrichment	Enable from day one — retrofitting is expensive
Build propagation	Standard K8s labels (`app.kubernetes.io/*`) — no custom tooling needed
Segments	Replace Management Zones for Gen3 — use enriched fields as filters
Verification	Run DQL checks after every deployment change

Additional Resources

- Set up Dynatrace on Kubernetes (DT docs)
- Quickstart for K8s setup (DT docs)
- DynaKube parameters (DT docs)

- Dynatrace Operator Helm chart (Dynatrace GitHub)
- Dynatrace Operator releases (Dynatrace GitHub)
- Kubernetes monitoring overview (DT docs)
- Metadata enrichment (DT docs)
- Operator + DynaKube troubleshooting (DT docs)
- Segments (DT docs)
